

Instituto Tecnológico y de Estudios Superiores de Occidente

Reconocimiento de validez oficial de estudios de nivel superior según acuerdo secretarial
15018, publicado en el Diario Oficial de la Federación del 29 de noviembre de 1976.

Department of Mathematics and Physics
Master of Data Science



Thesis Title

THESIS to obtain the **DEGREE** of
MASTER OF DATA SCIENCE

A thesis presented by:
Joey Ramone

Thesis Advisors:
Dr. Willie E. Coyote
Dr. Homer J. Simpson

Tlaquepaque, Jalisco, January, 2021

Instituto Tecnológico y de Estudios Superiores de Occidente

Reconocimiento de validez oficial de estudios de nivel superior según acuerdo secretarial 15018, publicado en el Diario Oficial de la Federación del 29 de noviembre de 1976.

Department of Mathematics and Physics Master of Data Science Approval Form

Thesis Title: **Thesis Title**

Author: **Joey Ramone**

Thesis Approved to complete all degree requirements for the Master of Science Degree in Data Science.

Thesis Advisor, **Dr. Willie E. Coyote**

Thesis Co-Advisor, **Dr. Homer J. Simpson**

Thesis Reader, **Dr. Frederick Flintstone**

Thesis Reader, **Dr. Farrokh Bulsara**

Academic Advisor, **Dr. Maryam Mirzajani**

Tlaquepaque, Jalisco, January, 2021

Thesis Title

Joey Ramone

Abstract

Here

Thesis Title

Joey Ramone

Resumen

En esta página debes escribir el abstract pero en español.

Contents

	Page
1 Introduction	19
1.1 Learning from Graph-Structured Data	19
1.2 Objectives and Scope	20
1.3 Structure of the Document	22
2 Learning Tasks on Graphs	23
2.1 Node-Level Tasks	24
2.2 Edge-Level Tasks	25
2.3 Graph-Level Tasks	26
Bibliography	27

List of Figures

	Page
1.1 Examples of graph-structured data.	20
2.1 The three levels of graph learning tasks.	23
2.2 Node-level tasks.	24
2.3 Edge-level tasks.	25
2.4 Graph-level tasks.	26

List of Tables

Page

Dedicated to:

Dedicado a:

Escribir la dedicatoria pero en español.

1 Introduction

Contents

1.1	Learning from Graph-Structured Data	19
1.2	Objectives and Scope	20
1.3	Structure of the Document	22

1.1 Learning from Graph-Structured Data

Data in machine learning often comes in a few familiar forms. Most datasets are tabular: a table of rows and columns in which each row is an independent example described by the same fixed set of features. Other data is organized as images, that is, grids of pixels, or as sequences such as text and audio, whose elements follow one another in a specific order. What these forms share is regularity: each has a fixed, well-defined structure that repeats across every example.

Many real-world systems do not fit any of these shapes. A molecule is a set of atoms connected by chemical bonds, and neither the number of atoms nor the way they are connected is the same from one molecule to the next. A social network is a set of people connected by relationships, where each person may be linked to a different number of others. A citation network is a set of papers that reference one another, each citing a different set of others. These are not tables, images, or sequences; they are networks of entities and the relations between them (Figure 1.1). Data of this kind is naturally described by a *graph*: a set of entities, called *nodes*, together with a set of connections between them, called *edges*. Both nodes and edges can carry their own data, for example in a molecule, each node is an atom labeled with its element (carbon, hydrogen, oxygen, and so on) and each edge is a bond of a particular type (single, double, and so on); in a social network, the nodes are people and the edges are the relationships among them.

Rather than treating a domain as a collection of isolated data points, we describe it in terms of entities and the relationships among them, which means there is an underlying graph whose structure ties those

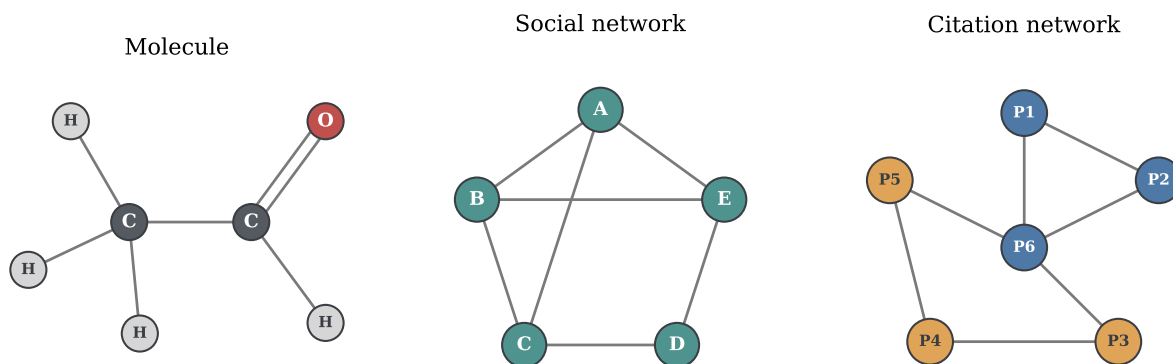


Figure 1.1: Three domains represented as graphs. A molecule, whose nodes are atoms labeled by their element and whose edges are bonds (single or double); a social network, whose nodes are people and whose edges are relationships; and a citation network, whose nodes are papers (colored by topic) and whose edges are citations between them. In each case the entities and their connections, not a table, grid, or sequence, define the data.

entities together. Seen this way, the graph is not merely one possible encoding of a molecule or a social network but a faithful representation: it records exactly which entities exist and how they relate, without forcing the data into a shape it does not have. A wide range of domains exhibit this kind of relational structure, and modeling them explicitly as graphs, rather than ignoring the connections, can lead to more accurate predictions. Data organized in this way is called *graph-structured data*.

Most deep learning models are tailored to the regular structures described above, particularly sequences and images laid out on a grid. Graphs are harder to work with. They have no fixed size, since different graphs contain different numbers of nodes. They have no fixed layout: a pixel in a grid has a clear sense of place, with neighbors above, below, and to either side, whereas a node in a graph has no such spatial arrangement and may be connected to any number of others. And they have no natural ordering, with no first or last node and no canonical point from which to read the rest.

The goal, then, is a model that takes a graph as input and produces predictions about it, such as a label for each node, whether connections exist between pairs of nodes, or a single label for the graph as a whole. This raises the central question: how can a predictive model – concretely, a neural network – be designed to operate directly on a graph, respecting its variable size and its lack of order instead of discarding them? Neural networks built for this purpose are called *Graph Neural Networks*, hereafter abbreviated as GNNs.

1.2 Objectives and Scope

The general objective of this work is to provide a self-contained reference on GNNs for homogeneous graphs under the *message-passing* framework: a single document that develops the theory that motivates

and defines these models, presents representative architectures, and evaluates them through reproducible implementations.

Specifically, the work aims to:

- establish the foundations that motivate GNNs: how graphs represent relational data, why standard neural networks cannot learn from it directly, and the inductive biases a model on graphs must respect;
- develop the message-passing framework as the unifying formulation from which GNN architectures are derived;
- present representative architectures for the three standard learning tasks on graphs: the Graph Convolutional Network (GCN) for node classification, the Graph Auto-Encoder (GAE) for link prediction, and the Graph Isomorphism Network (GIN) for graph classification;
- implement the three architectures on standard benchmark datasets (Cora, Amazon Computers, and NCI1) using PyTorch Geometric, with reproducible implementations, and evaluate each against a multilayer perceptron baseline that ignores the graph, examining the predictive benefit of explicitly modeling the relational structure of the data;
- examine the principal limitations of GNNs under this framework, including *oversmoothing* and the ceiling on their expressive power.

Its contribution is the reference document itself: a single, reproducible source that develops the message-passing framework from first principles, presents representative GNN architectures, and serves as a foundational resource for studying the field.

The graphs we consider are treated mainly as *undirected*, meaning their connections carry no direction, and *simple*, meaning that at most one connection joins any pair of nodes and that no node is connected to itself. This choice reflects simplicity rather than a limitation of the framework: message passing has enough design space to accommodate *directed* graphs, *self-loops* (a node connected to itself), and even *multigraphs* (more than one connection between the same pair of nodes).

The graphs considered in this work are also *homogeneous*: they contain a single kind of node and a single kind of connection. This restriction is deliberate. The homogeneous setting is where the message-passing framework is most directly understood, and it is the foundation on which more elaborate models are built, including *heterogeneous* graphs, which carry several kinds of nodes and connections and require machinery beyond the scope of this work. Establishing the homogeneous case is a prerequisite for these extensions, which we

leave as future research. The formal definitions of these graph types are given later in the document.

1.3 *Structure of the Document*

This introduction has set out the motivation, objectives, and scope of the work. The remainder of the document is organized as follows.

The first chapters establish the foundations. Chapter 2 surveys the tasks that graph learning addresses at the node, edge, and graph level, together with the practical applications that motivate the rest of the work. Chapter ?? establishes the formal definitions and notation for graphs used throughout. Chapter ?? reviews the multilayer perceptron and shows why standard neural networks are ill-suited to relational data. Chapter ?? develops the notion of inductive bias and the permutation invariance and equivariance that any graph layer must satisfy.

Chapter ?? builds on these foundations to derive the message-passing framework: the message, aggregation, and update functions, the receptive field that successive layers induce, the expressive power of the resulting models, and the architectural design space it enables.

Chapter ?? presents the three representative architectures within this framework: GCN for node classification, GAE for link prediction, and GIN for graph classification. Chapter ?? turns to practice, providing reproducible implementations of these architectures on the Cora, Amazon Computers, and NCI1 benchmarks in PyTorch Geometric, with the accompanying code available in a companion repository.

Chapter ?? examines the limitations of GNNs under the message-passing framework, and Chapter ?? synthesizes the work and outlines directions for future research.

2 Learning Tasks on Graphs

Contents

2.1	Node-Level Tasks	24
2.2	Edge-Level Tasks	25
2.3	Graph-Level Tasks	26

Once a domain has been represented as graph-structured data, a natural question arises: what machine learning tasks can be defined on a graph? A graph can be viewed at three levels of granularity: its individual nodes, its edges, and the graph as a whole. Each level gives rise to a family of prediction tasks: predicting a property of each node, predicting the existence or type of a connection between pairs of nodes, or predicting a property of the entire graph. These are known, respectively, as *node-level*, *edge-level*, and *graph-level tasks* (Figure 2.1), and they structure the architectures and implementations developed in the rest of this work.

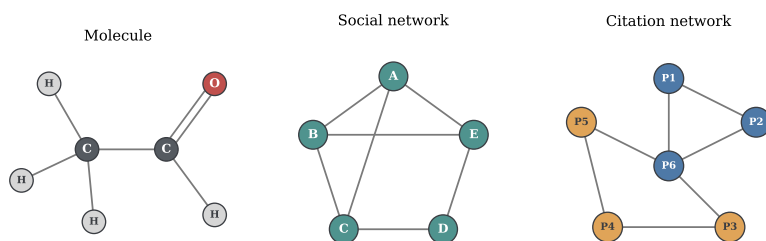
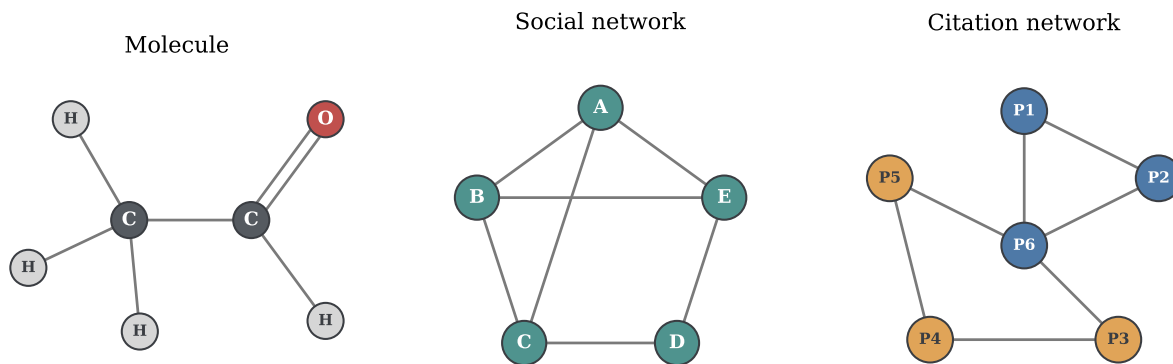


Figure 2.1: The three levels at which a prediction can be made on a single graph. A node-level task predicts a property of an individual node (highlighted); an edge-level task predicts a property of a connection between two nodes, such as whether it exists (dashed); and a graph-level task predicts a single property of the entire graph (framed). The same object supports all three.

These three levels correspond to the standard categories of graph learning tasks, but they do not exhaust the field. Other settings include *community/subgraph-level tasks*, which focus on groups of related nodes, and *graph generative tasks*, which aim to generate new graphs, such as candidate molecular structures with desirable properties. Although these settings rely on the same representational principles introduced here, they lie outside the scope of this work, which focuses on the three standard task levels.



The following sections examine each of these levels in turn and present representative applications that motivate them.

2.1 Node-Level Tasks

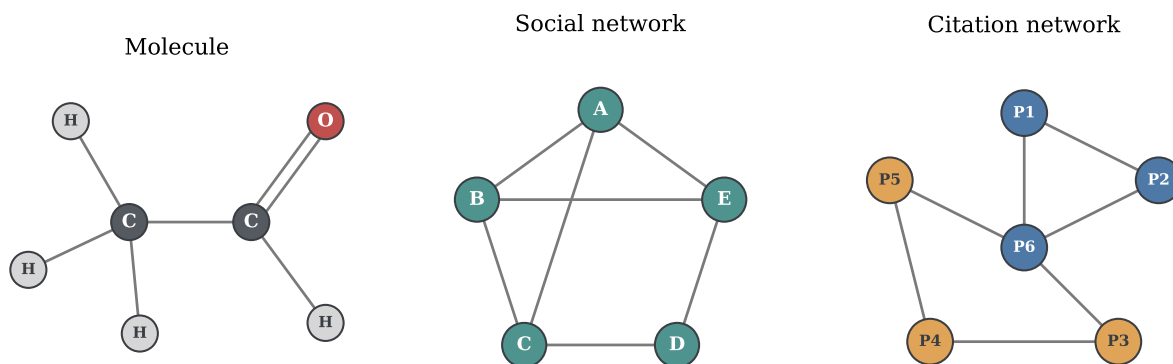
A *node-level task* asks for a prediction about each individual node of a graph, drawing on both the node's own attributes and its position within the surrounding structure. The most common instance is *node classification*: assigning each node to one of a set of categories. A representative setting is a citation network, in which the nodes are scientific papers, the edges are citations between them, and the goal is to infer the topic of each paper (Figure 2.2). A paper's own text is informative, but so is its neighborhood: papers tend to cite others on related subjects, so the topics of a paper's neighbors are strong evidence for its own. Node classification exploits exactly this, combining what a node contains with what surrounds it. The same pattern appears whenever entities in a network must be categorized.

The target need not be a category. A graph representation of a protein, for example, can be constructed where nodes represent amino acid residues and edges connect residues that are close in space. One may predict a discrete property of each residue, such as its functional role, or a continuous property, such as its position in three dimensions, a formulation that underlies recent advances in protein structure prediction¹. A node-level task can therefore be classification or regression, depending on whether the per-node target is categorical or numerical.

The same framing extends to economic settings. In a financial network whose nodes are accounts and whose edges are transactions between them, fraud detection amounts to flagging each account as legitimate or fraudulent: a node classification problem in which

Figure 2.2: Left: the node classification task in the abstract. Some nodes carry a known label (shown by color) while others are unknown (marked with a question mark); the model predicts a label for every node from its attributes and the surrounding structure. Right: a concrete instance as a citation network, where each node is a scientific paper and the predicted label is its topic.

¹ John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873): 583–589, 2021. DOI: 10.1038/s41586-021-03819-2



fraudulent behavior is often more evident in the surrounding network of connections than in any single account considered in isolation.

2.2 Edge-Level Tasks

An *edge-level task* focuses on the relationships between entities rather than the entities themselves. The central example is *link prediction*: given the edges already present in a graph, predict which further edges are likely to exist. Recommendation is one of the the most familiar application (Figure 2.3). In its simplest form, a network records products that are frequently bought together, with the nodes being products and edges indicating co-purchases. Predicting missing edges in this network amounts to identifying product pairs that are likely to be associated, even if they have not yet been observed together. A richer version introduces two kinds of entity, users and products, with an edge representing interactions such as purchases or ratings; here, link prediction becomes the recommendation problem itself, anticipating which products a user will want next.

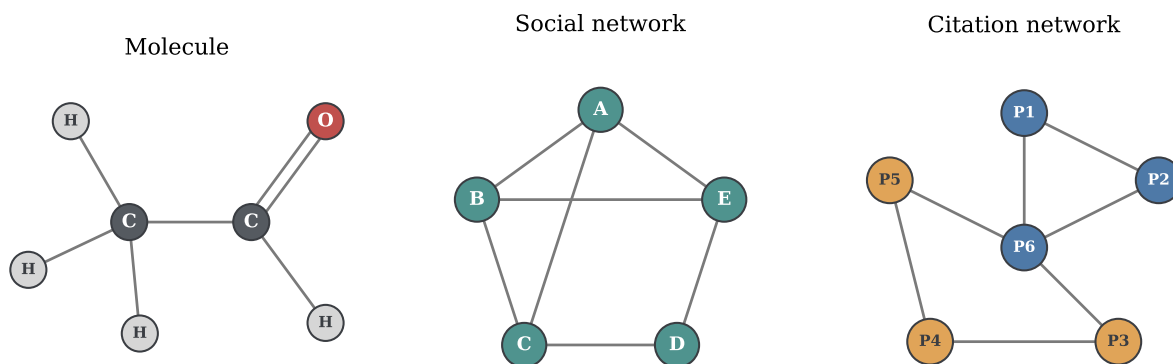
What makes the graph formulation effective is that it learns from the relational structure rather than treating each entity in isolation. A recommender system that treats products only by their individual attributes, ignoring the network of interactions around them, discards precisely the signal that reveals which items behave similarly; methods that propagate information along the graph structure consistently outperform such approaches, a principle that underlies graph-based recommender systems deployed at web scale ².

Link prediction extends well beyond commerce. Given a pair of drugs, one may predict whether taking them together produces an adverse interaction, by placing both within a larger biological network and asking whether a particular kind of connection should join them ³.

Figure 2.3: Left: the link prediction task in the abstract. Solid edges are observed connections; dashed edges marked with a question mark are candidate connections whose existence the model must predict. Right: a concrete instance as a recommendation setting, where the two kinds of entity (users, drawn as squares, and products, drawn as circles) are joined by observed interactions, and the task is to predict further user-product links.

² Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 974–983, 2018. DOI: 10.1145/3219819.3219890

³ Marinka Zitnik, Monica Agrawal, and Jure Leskovec. Modeling polypharmacy side effects with graph convolutional networks. *Bioinformatics*, 34(13):i457–i466, 2018. DOI: 10.1093/bioinformatics/bty294



The same formulation is also used to complete *knowledge graphs*, large networks of facts in which entities are connected by relations: many true relations are simply unobserved, and link prediction proposes the most plausible ones.

2.3 Graph-Level Tasks

A *graph-level task* treats an entire graph as a single example and predicts a property of the whole. The representative task is *graph classification*: assigning a label to a complete graph (Figure 2.4). Here the data is not one large network but a collection of many separate graphs, each with its own label, and the model must summarize the structure and attributes of an entire graph into a single prediction. Molecular property prediction is the canonical example. A molecule is naturally a graph, with atoms as nodes and chemical bonds as edges, and many prediction tasks about it are graph-level: whether a compound is toxic, or whether it is active against a given biological target. Because molecules vary in size and structure, this is exactly the regime that calls for a model able to handle graphs of varying structure and return one prediction per graph.

This capability feeds directly into drug discovery, where large libraries of candidate compounds are screened to prioritize the few worth synthesizing and testing in the laboratory, a process in which an accurate graph-level predictor can save considerable experimental effort. The same idea applies in materials science, where the atomic structure of a material is represented as a graph and a graph-level model predicts a physical property such as its stability or its mechanical strength.

Across all three levels, the same underlying object, the graph, is used in different ways, with the level of the task determining what is predicted and how a model must process the structure to predict it.

Figure 2.4: Left: the graph classification task in the abstract. The data is a collection of distinct graphs, differing in size and shape, each assigned a single label for the graph as a whole. Right: a concrete instance as molecular property prediction, where each graph is a molecule (atoms as nodes, bonds as edges) labeled by a property such as being active or inactive against a target.

Bibliography

John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstern, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596 (7873):583–589, 2021. DOI: 10.1038/s41586-021-03819-2.

Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 974–983, 2018. DOI: 10.1145/3219819.3219890.

Marinka Zitnik, Monica Agrawal, and Jure Leskovec. Modeling polypharmacy side effects with graph convolutional networks. *Bioinformatics*, 34(13):i457–i466, 2018. DOI: 10.1093/bioinformatics/bty294.